

CASPER: Carbon-Aware Scheduling and Provisioning for Distributed Web Services

Abel Souza, Shruti Jasoria, Basundhara Chakrabarty, David Irwin, Prashant Shenoy
University of Massachusetts Amherst
USA

Alexander Bridgwater, Axel Lundberg, Filip Skogh, Ahmed Ali-Eldin
Chalmers University of Technology
Sweden

ABSTRACT

There has been a significant societal push towards sustainable practices, including in computing. Modern interactive workloads such as geo-distributed web-services exhibit various spatiotemporal and performance flexibility, enabling the possibility to adapt the location, time, and intensity of processing to align with the availability of renewable and low-carbon energy. An example is a web application hosted across multiple cloud regions, each with varying carbon intensity based on their local electricity mix. Distributed load-balancing enables the exploitation of low-carbon energy through load migration across regions, reducing web applications carbon footprint. In this paper, we present CASPER, a carbon-aware scheduling and provisioning system that primarily minimizes the carbon footprint of distributed web services while also respecting their Service Level Objectives (SLO). We formulate CASPER as an multi-objective optimization problem that considers both the variable carbon intensity and latency constraints of the network. Our evaluation reveals the significant potential of CASPER in achieving substantial reductions in carbon emissions. Compared to baseline methods, CASPER demonstrates improvements of up to 70% with no latency performance degradation.

CCS CONCEPTS

• Computer systems organization → Cloud computing; Special purpose systems.

ACM Reference Format:

Abel Souza, Shruti Jasoria, Basundhara Chakrabarty, David Irwin, Prashant Shenoy and Alexander Bridgwater, Axel Lundberg, Filip Skogh, Ahmed Ali-Eldin. 2023. CASPER: Carbon-Aware Scheduling and Provisioning for Distributed Web Services. In *THE 14th international Green and Sustainable Computing Conference (IGSC '23)*, October 28–29, 2023, Toronto, ON, Canada. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3634769.3634812>

1 INTRODUCTION

In recent years, the global focus on sustainability and environmental responsibility has brought renewable energy to the forefront of the discussions on energy systems, leading to an increased focus on reducing the carbon footprint of cloud platforms in both research and industry [16, 17, 29, 31, 38]. Although there has been

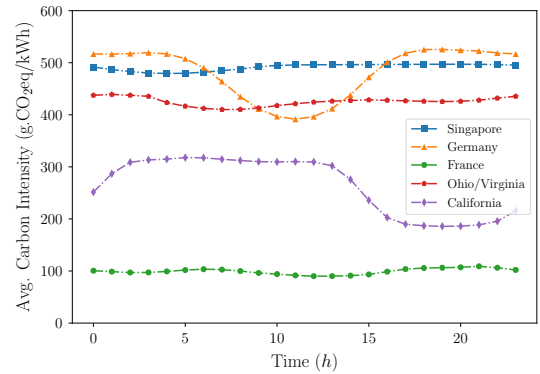


Figure 1: Grid carbon intensity in 2022 across six distinct cloud regions showing 6× spatial variations.

substantial progress in improving efficiency, today’s datacenter infrastructures consume around three to five percent of electricity worldwide and in ten years, five times as much [5, 15, 20]. These estimations may be lower than reality, as the growth of computing demand has been increasing exponentially for decades [7]. Cloud datacenters have mainly relied on enhancements in energy efficiency, which is unlikely to lead to significant reductions in carbon emissions as modern datacenters have already achieved high levels of optimization in energy efficiency. For instance, the Power Usage Effectiveness (PUE), a measurement of the total operational efficiency of most datacenters, is already near the optimal value of 1.0. More importantly, these trends are positioning cloud platforms as one of the largest contributors to global emissions [7]. Therefore, while energy efficiency improvements are important, they will be insufficient to counterbalance the rising energy consumption from the rapidly growing demand for cloud services. To effectively reduce carbon emissions, cloud platforms must shift their focus towards low-carbon energy sources. This entails harnessing energy derived from renewable sources such as wind, solar, hydro, nuclear, geothermal, and other sustainable alternatives.

To reduce cloud platforms’ carbon emissions, many have suggested leveraging computing workloads’ spatial and temporal flexibility, which is often significant, to dynamically shift the location and time of execution to better align with when and where low-carbon energy is available. Yet, despite the prominence of such simple carbon-aware spatiotemporal workload shifting as an abstract idea, prior work has only quantified its benefits in specific settings such as batch workloads. Web applications, in particular, serve as an excellent case for exploring the untapped potential of carbon-aware computing. These applications are typically distributed across multiple cloud servers located in different regions worldwide. Traditional approaches reduce latency by forwarding user requests to the geographically closest replica server, reducing

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

IGSC '23, October 28–29, 2023, Toronto, ON, Canada

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1669-0/23/10
<https://doi.org/10.1145/3634769.3634812>

load times and offering a better user experience. On the other hand, different cloud regions have varying carbon costs associated with their electricity sources, leading to different carbon footprints for user requests depending on which replica server services them. Consequently, optimizing the scheduling of user requests with respect to the carbon costs associated with different replicas presents an intriguing opportunity to make web applications more sustainable.

While renewable energy still continues to be unreliable due to its dependence on natural factors, web applications can still benefit from it without sacrificing performance. For instance, the inherent fault tolerance achieved through replication and load-balancing mechanisms can safeguard web applications against the intermittency and unpredictability of renewable energy sources [23]. By ensuring that replicas are spread across diverse regions and backed by different energy sources, these applications can maintain high availability while capitalizing on the potential of cleaner energy. However, although cloud providers maintain the information about the energy supply powering their servers, it is not readily available at a software level to applications [29]. Consequently, resource provisioners and load balancers cannot leverage this information to optimize the carbon efficiency of web workloads. Providing these systems with more visibility into the carbon footprint across data-centers enable the design of heuristics to provision resources and schedule workloads towards replicas with low carbon intensities while respecting applications Service Level Objectives (SLOs).

As such, we present CASPER, a carbon-aware scheduler and provisioner for distributed web applications. We assume a setting where resource provisioning and load balancing ought to happen in concert to minimize emissions while meeting application SLO targets. This is formulated as a multi-objective optimization problem, addressing both the carbon footprint resulting from server provisioning and the latency caused by load balancing. We evaluate the performance of spatial server provisioning and geo-distributed request scheduling for distributed web applications by implementing CASPER as a Kubernetes scheduler and submitting it to a real web workload. Our results highlight CASPER's significant potential in achieving considerable reductions in carbon emissions while meeting all latency constraints. In comparison to baseline methods, our approach demonstrates enhancements of up to 70% without compromising latency performance. We release CASPER as an open-source tool that can perform carbon optimizations for their distributed web applications: <https://github.com/carbonfirst/casper>

2 BACKGROUND

This section provides background on the grid carbon intensity, cloud model, and carbon-aware workload optimizations.

Carbon Intensity. The electric grid relies on a combination of generation sources to meet the demand for electricity, and include fossil fuel-based generators using coal or natural gas, low-carbon sources like hydro, wind, and solar, as well as non-carbon sources such as nuclear. Since electricity demand fluctuates throughout the day and follows diurnal patterns, the mix of generation sources and their relative proportions also vary over time. It is worth noting that renewable sources — such as wind and solar — are intermittent, which further impacts the overall generation mix. The carbon

intensity (CI) of electricity supply, measured in grams of CO₂ equivalent per watt or g-CO₂eq/kWh, represents the average weighted carbon intensity of the generation sources used at any given moment. As fossil-based sources have high, and renewable sources have low or zero carbon weights, the average CI depends on the proportion of each source in the overall generation mix. Figure 1 illustrates the average carbon intensity of the grid electricity over the 2022 period for six different geographical regions, revealing significant variations across locations. On the vertical axis, the carbon intensity exhibits spatial variations between regions, while the horizontal axis presents temporal variations within regions. As shown, France has the lowest carbon-intensity due to its reliance on nuclear power, while Germany and Singapore have the highest intensities due to their reliance on fossil fuels. However, regions like California and Germany have higher temporal variability due to increasing penetration of renewables. These variations imply that the carbon footprint of a job can vary by up to 40% depending on whether it is executed during a high or low carbon-intensity period. Moreover, they indicate that executing the same job in different cloud regions can lead to a 6-8x variation in emissions. This underscores the potential for techniques that strategically schedule workloads on clusters based on the current and projected carbon intensity of grid electricity. While cluster managers can leverage temporal variations by aligning job execution with low carbon periods [29, 36], we focus on exploiting spatial characteristics that involves distributing workloads across regions with both low carbon intensity and sufficient latency performance. Finally, our work concentrates on scheduling techniques aimed at reducing scope 2 emissions as defined by the GHG (Greenhouse Gas) protocol [26], in which the majority of operational emissions are attributed to energy consumption (including scope 1). We do not consider embodied emissions (scope 3).

Workload Flexibility. The potential for a job to reduce emissions is a function of its type – batch or interactive –, memory state, and the network latency and bandwidth across locations. Additionally, there may be regulatory constraints, such as HIPAA [3] and GDPR [12], that prevent spatially shifting a job outside of a specific country, region or jurisdiction. While batch jobs such as AI and machine learning often have flexible completion times and can accommodate temporal variations, interactive workloads have strict low-latency requirements and limited temporal flexibility. This is especially true in web-services environments where requests pass through multiple microservices before a response is produced. For instance, load balancing tools enable modern workloads with the ability to shift their execution location to minimize latency and improve service availability [30]. These techniques work mostly with workloads that have lightweight memory states and which do not need to transfer data around locations. In this study, we consider lightweight web-requests – specifically HTTP requests –, that can be seamlessly processed across various locations. These requests have latency requirements that need to be limited within a maximum threshold. Given that numerous services are highly optimized for latency, minor deviations within a specified target are unlikely to impact the overall user experience, and can enable the exercise of spatial shifting to optimize for carbon.

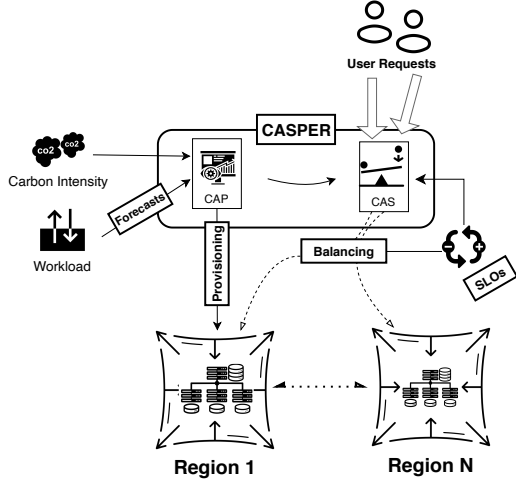


Figure 2: CASPER: CAP and CAS provision and coordinate user workloads.

3 SYSTEM DESIGN AND IMPLEMENTATION

This section outlines the design and architecture of CASPER, along with its key components.

3.1 Architecture

CASPER is designed as a modular system that can be integrated into any existing distributed resource manager. Figure 2 illustrates the overall system architecture, highlighting its two main components: the Carbon-Aware Provisioner (CAP) and the Carbon-Aware Scheduler (CAS). CASPER includes various components for interfacing with interactive jobs, such as the resource manager, monitoring, and the carbon-aware load-balancing and scheduling policies.

Carbon-Aware Provisioner. CAP acts as an intelligent provisioner that analyzes the inter-regional network latency, the region’s (variable) carbon intensity, and the expected application’s workload. Besides reducing carbon, CAP provides operators with an important estimator: the optimal number of servers needed in each region such that the expected workload is correctly handled for each time period and lowest carbon intensity. This intuition leads us to formalize this provisioning problem as a *multi-objective formulation*.

$$\min_x \quad \alpha \sum_j I_j \sum_i x_{ij} + (1 - \alpha) \sum_j s_j \quad (1a)$$

$$\text{s.t.} \quad \sum_i x_{ij} \leq s_j c_j \quad (1b)$$

$$\sum_j s_j \leq K \quad (1c)$$

$$x_{ij} (\ell_{ij} - L) \leq 0 \quad (1d)$$

$$\sum_{i,j} x_{ij} = \mathbb{E}[\lambda_i], \forall i, j \quad (1e)$$

$$\alpha \in [0, 1] \quad (1f)$$

$$\bar{x}_j s_j = 0 \quad (1g)$$

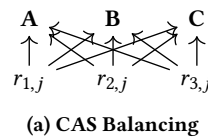
$$x_{ij}, s_j \in \mathbb{Z}_{\geq 0} \quad (1h)$$

Parameter	Description
x_{ij}	Requests redirected from region i to region j
$\bar{x}_j$	Requests not sent to region j
s_j	Number of servers in region j
n	Number of regions $\mathcal{R}$
I_j	Carbon intensity in region j
α	Normalized weight for the carbon intensity (in relation to number of servers s_j)
λ_i	Incoming request rate at region i
ℓ_{ij}	Expected latency from region i to j
c_j	Resource capacity of region j (in # of requests)
L_i	Maximum tolerated latency for a request
K	Maximum number of servers across all locations
t_j	Number of requests submitted to region j

Table 1: List of parameters used by CAP.

We present CAP’s formulation in Equation 1, and Table 1 describes all of its parameters. All indices i, j represent the set of available regions $\mathcal{R}$ for resource allocation, request processing and redirection capacities. We let $\bar{x}_j \in \{0, 1\}$ be the variable that represents requests that are not sent to a region j , i.e., $\sum_j x_{ij} = 0$. This constraint effectively means that if it is anticipated that region j won’t receive any requests, there should be no server allocation in that region. Moreover, x_{ij} represents the optimal count of requests from region i that is redirected to region j , while s_j is the number of servers provisioned in region j to handle incoming requests. Eq. 1a aims to minimize both the total carbon footprint of executing requests (Eq. 1b) and the cumulative number of servers s_j across all regions (Eq. 1c) such that the minimum latency target is guaranteed (Eq. 1d). Finally, the following assumptions are made. First, the problem is defined within the scope of minimizing carbon emissions while simultaneously adhering to application latency constraints. Second, since we consider cloud datacenters, we ignore issues regarding resource limits, although we do include a maximum amount of servers (Eq. 1c) that CASPER can provision. We also assume the load-balancers communication latency across regions (as seen in Figure 2) is negligible when compared to the requests’ average service (processing) times. Finally, the provisioner uses forecasts for carbon intensity [24] and hourly workload request rates that are expressed in terms of expected arrivals in region i (Eq. 1e).

3.2 Carbon Aware Scheduler



(a) CAS Balancing

$$r_{i,j} = \frac{\sum_i x_{ij}}{\sum_i \sum_j x_{ij}} t_j$$

(b) Weight Factors

Figure 3: Illustration of CAS and weight calculation.

Figure 3 shows how the Carbon Aware Scheduler (CAS) distributes requests between regions x_{ij} , which denotes the load of requests that need to be redirected from region i to j . It uses a vector as shown by Equation 3(b) to model each region r ’s weight, following the timely estimates obtained from CAP (e.g., hourly). We implement CAS as a load-balancer module in CASPER, whereas local

Geographical Region	AWS Region
California	US-West-1
Virginia	US-East-1
Ohio	US-East-2
Germany	EU-Central-1
France	EU-West-3
Singapore	AP-SouthEast-2

Table 2: AWS Regions used in the evaluation.

incoming requests are redistributed across all regions according to their proportional weights. More importantly, CAS ensures that unforeseen workload events — e.g., load spikes, not accounted for in CAP’s optimization — are effectively handled as best as possible.

3.3 Implementation

CASPER is implemented as a Kubernetes (K8S) scheduler with $s < K$ workers, each representing a cloud region. Each deployment comprises of a single K8S pod that runs the application. A prototype has been developed to emulate the operations of a Wikipedia-like service across six distinct AWS regions, as detailed in Table 2. These regions are selected as the closest regions to the original Wikimedia servers [14].

CAP. CASPER’s provisioner is developed using Python, while the optimizations are solved using the PuLP library [25], an interface to the Coin-or branch and cut (CBC) solver [28]. The region wise server deployment array obtained as an output of CAP is used to scale the size of each regions using server collected metrics. Additionally, CAP computes the optimal request distribution matrix, which is forwarded to CAS.

CAS. The scheduler coordinates a set of load balancers, one per-region, to implement its logic. It timely forwards incoming requests to the appropriate regions following the weights derived in the CAP’s optimal request distribution matrix. Traefik [35] is used to establish the cluster’s load balancer layer, creating a HTTP proxy for every region to receive and forward requests by routing the traffic to one of the corresponding backend regions based on the hourly weights calculated by CAP.

4 EVALUATION

In this section, we first discuss the real-world application, workload, carbon, and network traces that are utilized to evaluate CASPER. Then, we discuss the policies briefly introduced in the previous section. Finally, we demonstrate and quantify the trade-offs between carbon savings and latency performance for various targets.

4.1 Setup

Infrastructure. CASPER runs on Ubuntu Linux 20.04, and it consists of a control plane and worker nodes. The cluster comprises 16 servers with 16-cores Intel Xeon processors and 32GB of memory. Each node runs a Kubernetes deployment representing one region. For intra-cluster communication, an overlay network is created using Flannel [6].

Application. To evaluate CASPER, we deploy Kiwix [9], a platform to host and distribute compressed versions of the Wikipedia [10]. Specifically, we load Kiwix with the pre-built version of the German

Wikipedia from May 2023, which comprises a total of 32 GB of content [11]. Requests are directed through the CAS load balancer, which interconnects all nodes in the cluster via a HTTP port.

Carbon Intensity. Figure 1 illustrates the carbon intensity data for all the aforementioned geographical regions (Table 2) at an hourly granularity. This data has been collected from Electricity Maps [8] for 2022.

Workload and Network Traces. We use the Wikimedia’s dataset [13] covering six datacenters across the USA, Europe, and Asia. For each region, the dataset includes the request rates (requests per-second) and datacenter hourly utilization covering 2022. Since two of the AWS regions do not match those from Wikimedia’s — i.e., Netherlands and Texas —, we select the two closest AWS regions i.e., Germany and Ohio (Table 2). Average latency data (in milliseconds) across all AWS regions are obtained from Cloudping [1] for 2022.

Telemetry. Each region’s load-balancer exports their service-level metrics, specifically the total count of HTTP requests served by each endpoint and their associated service time. To calculate the carbon cost of request execution, this metric is multiplied by the region’s current hour’s carbon intensity.

Policies. We conduct a comprehensive evaluation of CASPER throughout the entire year of 2022. The parameters for CAP are set as follows: $n = 6$ (representing the AWS regions), $\alpha = 0.5$ (equal weights to both carbon and latency costs), $c_j = 100$ (one server can handle up to 1k requests), and $K = 500$ (global maximum number of servers). We also introduce several variations in the values of L_i (see below), which establish the maximum acceptable latency for each request. CAP runs at the beginning of every hour to determine the provisioning of servers at each location. The CAS weights are then calculated based on the output of CAP. To assess the system’s performance, we execute a real workload simulation spanning 24 hours. Metrics are collected at 10-minute intervals and aggregated at the end of each hour. We conduct evaluations using the following policies:

- (1) **Latency.** This simulation serves as the baseline scenario without any carbon optimization, where requests are solely served based on the lowest latency, i.e., locally in the originating region, without any load balancing.
- (2) **Carbon-L Policies.** These runs focus on carbon optimization with various latency L threshold guarantees, ranging from 20 to 500ms. This approach involves a trade-off in terms of performance, as requests can be redirected as long as the latency requirements remain below L ms.

These implementations strictly follow Mediawiki’s operations, in particular the Latency policy that adheres to their stringent latency requirements [14]. Among the carbon-aware policies, we set one with threshold of $L = 500$ ms as it represents the most flexible response time across all regions. In this particular setting, the carbon cost of execution is minimized by irrestrictive redirections that can reach very distant, lower carbon regions capable of accommodating the redirected requests.

Workload Generation. A sample of the workload is represented in Figure 4(a), with incoming requests in all regions. Each hour is divided into timesteps, and the request rate for each timestep is

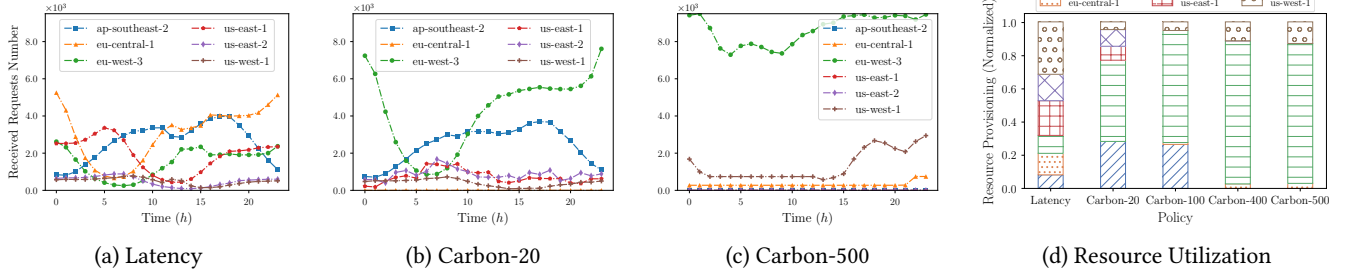


Figure 4: Redirection rate (a, b and c) and resource provisioning per-region and policies (d): Provisioning tends to increase in greener nearby regions.

selected from a set of values that follow an exponential distribution. Parameters to generate the distributions are selected such that the upper limit of the generated values is approximately $1.5\times$ the request rate for the hour.

4.2 Results

Effects on Request Redirection. Figures 4(a)-(c) present workload redirection results for *Latency*, *Carbon-20*, and *Carbon-500*. As shown in Figure 1, Zone "eu-west-3" (France) has the lowest carbon intensity, followed by "us-west-1" (California), "us-east-[1,2]" (Ohio/Virginia), "eu-central-1" (Germany), and "ap-southeast-2" (Singapore). Figure 4(a) simply shows the original workload, where no redirection happens. Notably in Figure 4(b), due to close proximity and low carbon intensity, CASPER redirects as many requests as possible from Germany towards France. And due to the latency constraints (20ms), Ohio and Virginia cannot induce savings. This behavior is more evident in 4(c): As the latency constraint is relaxed (500ms), France and California receive as many requests as possible from all regions. However, eu-west-3 reaches capacity at various moments, triggering CASPER to forward load to California (us-west-1).

represents the original provisioning with no redirections. As the latency constraints increase, the CASPER initiates re-provisioning of servers from the German ("eu-central-1") region to France due to its lower carbon and lower network latency. Under *Carbon-100*, a significant portion of the Ohio and Virginia workloads are redirected exclusively to France, as the 100ms latency requirement can be fulfilled. It is worth noting that requests originating from Singapore are only directed to greener locations under the *Carbon-400* policy. This limitation arises from the end-to-end latency from Singapore to any other region surpassing the 100ms threshold. Moreover, the capacity in France reaches its limit under the *Carbon-400* policy, prompting redirections towards California, in addition to few towards Germany to meet the latency requirements.

Effects on Carbon and Latency. Figures 5(a)-(d) present a comparative analysis of all policies. Figures 5(a) and (c) clearly illustrate the primary tradeoff of CASPER, wherein *the relaxation of latency constraints leads to an increased potential for emissions reduction*. The *Latency* policy, despite achieving an average response time as low as 6ms, exhibits the highest carbon emissions due to requests remaining localized in high-intensity regions such as Germany and Singapore. Notably, *Carbon-20* demonstrates that even small relaxations in latency constraints can result in a 25% carbon reductions. *Carbon-100* achieves a 37% reduction, while *Carbon-400* reaches a point of diminishing returns with a 70% reduction, similar to *Carbon-500* which represents an unrestricted carbon optimization scenario. Moreover, Figures 5(b) and (d) display the hourly variations in average latency and emissions, respectively. In comparison to the *Latency* policy, *Carbon-20* shows a minimal increase in latency of 6ms while simultaneously reducing emissions. *Carbon-100* through *Carbon-500* exhibit latency increases ranging from 5-16 $\times$, although delivering the most substantial reductions. Finally, it is important to note that results would change with other α values. This is primarily due to the fact that CASPER would redirect requests differently due to the trade-off between carbon emissions and the number of servers needed to satisfy latency SLOs. Specifically, as α increases, CASPER would redirect more requests to greener regions at the cost of latency because this would reduce the number of servers in browner regions. In contrast, as α decreases, CASPER would prioritize latency, opting to handle requests locally despite the carbon costs of setting additional servers.

5 RELATED WORK

Recent efforts have concentrated on harnessing the flexibility in energy demand for diverse workloads to diminish their carbon

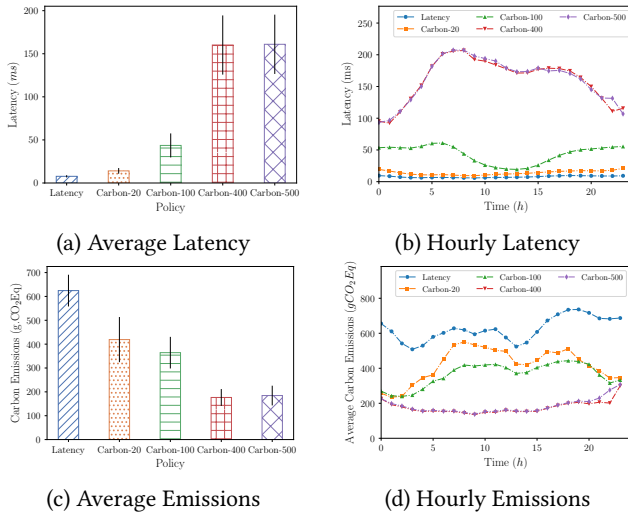


Figure 5: Latency and carbon tradeoffs across policies.

Effects on Resource Provisioning. Figure 4(d) illustrates the resource provisioning across the six AWS regions. The *Latency* policy

footprint by leveraging the temporal and spatial flexibility of computing [4, 16, 17, 21, 22, 27, 29, 32, 33, 36, 38]. Treehouse [2] proposes a software-centric approach to reduce the carbon intensity of datacenter computing by making energy and carbon visible at the application layer. CADRE focuses on carbon-aware data replication to reduce overall carbon footprint, while leveraging load flexibility and interactions with the electricity market to minimize carbon emissions [39]. [37] investigates the potential of shifting computational workloads to less carbon-intensive periods based on the fluctuating carbon intensity of energy supply. [19] introduces a low-carbon extension to the Kubernetes scheduler, sorting cloud regions by carbon intensity and migrating workloads to regions with low carbon cost. However, the proposed framework is evaluated primarily for batch jobs. On the other hand, numerous works have employed integer programming techniques to devise new techniques for low-carbon scheduling [34]. Carbon-aware geo-distributed scheduling is particularly relevant for Machine Learning (ML) workloads requiring long periods of execution [18]. [40] proposes Cucumber, an admission control policy that leverages load and energy forecasting techniques to determine scheduling strategies to use renewables. Unlike the previous works, CASPER is the first framework that seamlessly integrates server provisioning and request scheduling for a geo-distributed web application, with a particular focus on interactive web requests.

6 CONCLUSION AND FUTURE WORK

This paper introduced CASPER, a carbon-aware scheduler and provisioner designed for distributed web applications. At the heart of CASPER lies a multi-objective optimization that minimizes both resources and latency, introducing a novel method to control the load balancing of web applications. We observe substantial savings in the carbon footprint, reaching up to 70% with controllable and negligible losses in performance while meeting all SLOs. CASPER represents a crucial advancement in carbon-aware schedulers for distributed and geo-distributed applications. Further analysis and exploration of additional spatiotemporal carbon-aware strategies are warranted to enhance the system's efficiency. As a potential avenue for future work, the implementation of auto-scaling policies that continuously monitors resource utilization across regions to dynamically adapt allocations could be explored.

ACKNOWLEDGMENTS

We thank the reviewers for their valuable comments, which improved the quality of this paper, and Electricity Maps for access to their carbon-intensity data. This research is supported by NSF grants 2211302, 2211888, 2213636, 2105494, and Army contract W911NF-17-2-0196.

REFERENCES

- [1] Matt Adorjan. 2017. CloudPing. www.cloudping.co.
- [2] Thomas Anderson, Adam Belay, Mosharaf Chowdhury, Asaf Cidon, and Irene Zhang. 2023. Treehouse: A case for carbon-aware datacenter software. *ACM SIGENERGY Energy Informatics Review* 3, 3 (2023), 64–70.
- [3] HIPAA Compliance Assistance. 2003. Summary of the hipaa privacy rule. *Office for Civil Rights* (2003).
- [4] Noman Bashir, Tian Guo, Mohammad Hajiesmaili, David Irwin, Prashant Shenoy, Ramesh Sitaraman, Abel Souza, and Adam Wierman. 2021. Enabling Sustainable Clouds: The Case for Virtualizing the Energy System. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '21)*. Association for Computing Machinery, New York, NY, USA, 350–358. <https://doi.org/10.1145/3472883.3487009>
- [5] Gary Cook and David Pomerantz. 2015. Clicking clean: A guide to building the green Internet. *Greenpeace International, Tech. Rep.* (2015).
- [6] CoreOS. 2016. Flannel. <https://github.com/flannel-io/flannel>.
- [7] Peter J. Denning and Ted G. Lewis. 2017. Exponential Laws of Computing Growth. *Commun. ACM* 60, 1 (January 2017), 54–65.
- [8] ElectricityMap. 2022. ElectricityMap. <https://electricitymaps.com/>.
- [9] Emmanuel Engelhart and Renaud Gaudin. 2007. Kiwix. <https://kiwix.org>.
- [10] Emmanuel Engelhart, Tommi Makitalo, and Manuel Schneider. 2016. openZIM. <https://openzim.org>.
- [11] Emmanuel Engelhart, Tommi Makitalo, and Manuel Schneider. 2023. Latest Wikipedia zim dump. <https://download.kiwix.org/zim/wikipedia/>.
- [12] European Parliament and Council of the European Union. 2016. Regulation (EU) 2016/679 of the European Parliament and of the Council. <https://data.europa.eu/eli/reg/2016/679/oj>.
- [13] Wikimedia Foundation. 2023. Wikimedia's Grafana installation. <https://grafana.wikimedia.org/>.
- [14] Wikimedia Foundation. 2024. Wikimedia infrastructure. https://wikitech.wikimedia.org/wiki/Wikimedia_infrastructure.
- [15] C. Garcia. 2022. AKCP, The Real Amount of Energy A Data Center Uses. <https://www.akcp.com/blog/the-real-amount-of-energy-a-data-center-uses/>.
- [16] Udit Gupta, Mariam Elgamal, Gage Hills, Gu-Yeon Wei, Hsien-Hsin S. Lee, David Brooks, and Carole-Jean Wu. 2022. ACT: Designing Sustainable Computer Systems with an Architectural Carbon Modeling Tool. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 784–799.
- [17] Udit Gupta, Young Geun Kim, Sylvia Lee, Jordan Tse, Hsien-Hsin S. Lee, Gu-Yeon Wei, David Brooks, and Carole-Jean Wu. 2021. Chasing Carbon: The Elusive Environmental Footprint of Computing. In *HPCA*. ACM.
- [18] Kawsar Haghsheenas, Brian Setz, and Marco Aiello. 2022. CO2 Emission Aware Scheduling for Deep Neural Network Training Workloads. In *2022 IEEE International Conference on Big Data (Big Data)*. 1542–1549. <https://doi.org/10.1109/BigData55660.2022.10020544>
- [19] Aled James and Daniel Schien. 2019. A low carbon kubernetes scheduler. In *6th International Conference on ICT for Sustainability, ICT4S 2019 (CEUR Workshop Proceedings, Vol. 2382)*. CEUR-WS. 6th International Conference on ICT for Sustainability, ICT4S 2019 ; Conference date: 10-06-2019 Through 14-06-2019.
- [20] Nicola Jones. 2018. How to stop data centres from gobbling up the world's electricity. *Nature* 561, 7722 (2018), 163–167.
- [21] Adam Lechowicz, Nicolas Christianson, Jinhang Zuo, Noman Bashir, Mohammad Hajiesmaili, Adam Wierman, and Prashant Shenoy. 2024. The Online Pause and Resume Problem: Optimal Algorithms and An Application to Carbon-Aware Load Shifting. In *Proceedings of the ACM on Measurement and Analysis of Computing Systems (SIGMETRICS 2024)*. ACM, New York, NY, USA, 35 pages.
- [22] Liuzixuan Lin and Andrew A Chien. 2023. Adapting Datacenter Capacity for Greener Datacenters and Grid. In *Proceedings of the ACM International Conference on Future Energy Systems (e-Energy)*. ACM, New York, NY, USA, 200–213.
- [23] Diptyarop Maji, Ben Pfaff, Vipin PR, Rajagopal Sreenivasan, Victor Firoiu, Sreeram Iyer, Colleen Josephson, Zhelong Pan, and Ramesh K Sitaraman. 2023. Bringing Carbon Awareness to Multi-cloud Application Delivery. In *Proceedings of the 2nd Workshop on Sustainable Computer Systems*. 1–6.
- [24] Diptyarop Maji, Prashant Shenoy, and Ramesh K Sitaraman. 2022. CarbonCast: multi-day forecasting of grid carbon intensity. In *Proceedings of the 9th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*. 198–207.
- [25] Stuart Mitchell, Michael OSullivan, and Iain Dunning. 2011. PuLP: a linear programming toolkit for python. *The University of Auckland, Auckland, New Zealand* 65 (2011).
- [26] Green Gas House Protocol. 2024. GHG Protocol Scope 2 Guidance. <https://ghgprotocol.org/scope-2-guidance>.
- [27] Ana Radovanović, Ross Koningstein, Ian Schneider, Bokan Chen, Alexandre Duarte, Binz Roy, Diyue Xiao, Maya Haridasan, Patrick Hung, Nick Care, Saurav Talukdar, Eric Mullen, Kendal Smith, MariEllen Cottman, and Walfredo Cirne. 2023. Carbon-Aware Computing for Datacenters. *IEEE Transactions on Power Systems* 38, 2 (2023), 1270–1280.
- [28] Matthew J Saltzman. 2002. COIN-OR: an open-source library for optimization. *Programming languages and systems in computational economics and finance* (2002), 3–32.
- [29] Abel Souza, Noman Bashir, Jorge Murillo, Walid Hanafy, Qianlin Liang, David Irwin, and Prashant Shenoy. 2023. Ecovisor: A Virtual Energy System for Carbon-Efficient Applications. In *ASPLOS*. ACM, New York, NY, USA, 252–265.
- [30] Abel Souza, Alessandro Vittorio Papadopoulos, Luis Tomas, David Gilbert, and Johan Tordsson. 2018. Hybrid adaptive checkpointing for virtual machine fault tolerance. In *2018 IEEE International Conference on Cloud Engineering (IC2E)*. IEEE, 12–22.
- [31] Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2020. Energy and Policy Considerations for Modern Deep Learning Research. In *AAAI Conference on Artificial Intelligence (AAAI)*. ACM, New York, NY, USA, 13693–13696.
- [32] Thanathorn Sukprasert, Abel Souza, Noman Bashir, David Irwin, and Prashant Shenoy. 2023. Quantifying the Benefits of Carbon-Aware Temporal and Spatial

- Workload Shifting in the Cloud. *arXiv preprint arXiv:2306.06502* (2023).
- [33] Seyedali Tabaeiaghdaei, Simon Scherrer, Jonghoon Kwon, and Adrian Perrig. 2023. Carbon-Aware Global Routing in Path-Aware Networks. In *Proceedings of the ACM International Conference on Future Energy Systems (e-Energy)*. ACM, New York, NY, USA, 144–158.
 - [34] Samuel Trevino-Martinez, Rapinder Sawhney, and Oleg Shylo. 2022. Energy-carbon footprint optimization in sequence-dependent production scheduling. *Applied Energy* 315 (2022), 118949. <https://doi.org/10.1016/j.apenergy.2022.118949>
 - [35] Emile Vauge. 2016. Traefik. <https://traefik.io/traefik/>.
 - [36] Philipp Wiesner, Ilja Behnke, Dominik Scheinert, Kordian Gontarska, and Lauritz Thamsen. 2021. Let's Wait Awhile: How Temporal Workload Shifting Can Reduce Carbon Emissions in the Cloud. In *Proceedings of the 22nd International Middleware Conference (Middleware)*. ACM, New York, NY, USA, 260–272.
 - [37] Philipp Wiesner, Ilja Behnke, Dominik Scheinert, Kordian Gontarska, and Lauritz Thamsen. 2021. Let's wait awhile: How temporal workload shifting can reduce carbon emissions in the cloud. In *Proceedings of the 22nd International Middleware Conference*. 260–272.
 - [38] Carole-Jean Wu, Ramya Raghavendra, Udit Gupta, Bilge Acun, Newsha Ardalani, Kiwan Maeng, Gloria Chang, Fiona Aga, Jinshi Huang, Charles Bai, et al. 2022. Sustainable AI: Environmental Implications, Challenges and Opportunities. *Proceedings of Machine Learning and Systems* 4, 795–813.
 - [39] Zichen Xu, Nan Deng, Christopher Stewart, and Xiaorui Wang. 2015. Cadre: Carbon-aware data replication for geo-diverse services. In *2015 IEEE International Conference on Autonomic Computing*. IEEE, 177–186.
 - [40] Siyue Zhang, Minrui Xu, Wei Yang Bryan Lim, and Dusit Niyato. 2023. Sustainable AIGC Workload Scheduling of Geo-Distributed Data Centers: A Multi-Agent Reinforcement Learning Approach. *arXiv:2304.07948 [cs.AI]*